Maximal Marginal Relevance (MMR) in RAG

Balancing Query Relevance and Context Diversity

Detailed Notes – Agentic AI Bootcamp

Contents

1 Introduction **2**

2 The Redundancy Problem **2**

3 Ordinary Vector Retrieval **2**

4 The MMR Objective **2**

5 Worked Numerical Example **3**

6 The Role of λ **3**

7 When MMR Is Potentially Useful **4**

8 Embedding Dimension and Computational Cost **4**

9 Embedding Geometry **5**

10 Optimization Strategies **5**

11 MMR vs. Neural Reranking **6**

12 Practical Enterprise Example **6**

13 Key Takeaways **7**

14 One-Sentence Summary **7**

15 References **7**

1 Introduction

Maximal Marginal Relevance (MMR) is a retrieval diversification technique. In a RAG pipeline, ordinary Top-K similarity search may return several chunks that are individually highly relevant but largely repeat the same information. MMR tries to select chunks that are both relevant to the query and different from chunks already selected.

MMR = Relevance Reward − Redundancy Penalty

2 The Redundancy Problem

Consider the query: *"What are the company's policies for working remotely?"*

A vector search might return:

Rank	Chunk	Query similarity
1	A: Remote work eligibility	0.95
2	B: Remote work eligibility and manager approval	0.93
3	C: Remote employee requirements	0.90
4	D: International remote work	0.87
5	E: Office attendance requirements	0.82

A naive Top-3 strategy returns A, B, C. If those chunks overlap heavily, the LLM context wastes space repeating one topic while omitting other useful aspects. MMR asks: *"Is the next candidate relevant, and does it add information not already represented?"*

3 Ordinary Vector Retrieval

Let q be the query embedding and d_i the embedding of candidate chunk i. Cosine similarity is:

$$Sim(q, d_i) = \frac{q \cdot d_i}{\|q\|\|d_i\|}$$

Ordinary retrieval approximately selects the K chunks with the largest $Sim(q, d)$. This explicitly optimizes query similarity, not diversity among the selected results.

4 The MMR Objective

The standard MMR objective is:

$$\boxed{MMR(d) = \lambda Sim(q, d) - (1 - \lambda) \max_{d' \in S} Sim(d, d')}$$

where S is the set of chunks already selected and $\lambda \in [0, 1]$ controls the relevance–diversity trade-off.

The first term,

$$\lambda Sim(q, d),$$

rewards query relevance. The second term,

$$(1 - \lambda) \max_{d' \in S} Sim(d, d'),$$

penalizes a candidate that is highly similar to any selected chunk.

5 Worked Numerical Example

Assume pairwise candidate similarities are:

	A	B	C	D	E
A	1.00	0.96	0.91	0.30	0.20
B	0.96	1.00	0.94	0.25	0.15
C	0.91	0.94	1.00	0.35	0.25
D	0.30	0.25	0.35	1.00	0.40
E	0.20	0.15	0.25	0.40	1.00

Let $\lambda = 0.5$. Initially $S = \emptyset$, so A is selected because it has the highest query similarity:

$$S = \{A\}.$$

For B:

$$MMR(B) = 0.5(0.93) - 0.5(0.96) = 0.465 - 0.480 = \boxed{-0.015}.$$

For D:

$$MMR(D) = 0.5(0.87) - 0.5(0.30) = 0.435 - 0.150 = \boxed{0.285}.$$

Although B has greater query similarity, D wins because it is much less redundant. Now:

$$S = \{A, D\}.$$

For C, the maximum similarity to the selected set is $\max(0.91, 0.35) = 0.91$:

$$MMR(C) = 0.5(0.90) - 0.5(0.91) = \boxed{-0.005}.$$

For E, $\max(0.20, 0.40) = 0.40$:

$$MMR(E) = 0.5(0.82) - 0.5(0.40) = 0.410 - 0.200 = \boxed{0.210}.$$

Thus MMR selects:

$$\boxed{A, D, E}$$

instead of ordinary similarity retrieval's:

$$\boxed{A, B, C}.$$

6 The Role of λ

If $\lambda = 1$, then:

$$MMR(d) = Sim(q, d),$$

which reduces to ordinary similarity ranking.

A value such as $\lambda = 0.8$ strongly favors relevance. $\lambda = 0.5$ balances relevance and diversity. A low value such as $\lambda = 0.2$ strongly favors diversity and can select chunks that are different but insufficiently relevant.

> **MMR does not mean "maximize diversity." It means preserve relevance while discouraging unnecessary redundancy.**

7 When MMR Is Potentially Useful

MMR is particularly useful when:

- the corpus contains duplicate or overlapping documents;
- chunk overlap causes neighboring chunks to be near duplicates;
- multiple enterprise sources repeat the same policy or fact;
- the question is broad or has multiple aspects;
- the application has a limited context budget.

For example, for "What are the different ways customers can authenticate?", naive retrieval might return password policy, password reset, password expiration, and password requirements. MMR may encourage coverage of passwords, OAuth, SAML/SSO, API keys, and MFA.

MMR may be less useful for a very specific query where several closely related chunks are all genuinely needed. Diversification should therefore be evaluated rather than assumed to improve quality.

8 Embedding Dimension and Computational Cost

Embedding dimension does not itself create diversity. MMR's diversification comes from the redundancy penalty. Dimension does, however, affect the cost of similarity calculations.

For embeddings of dimension d, a dot product has work proportional to:

$$O(d).$$

If N candidates are considered and MMR greedily selects K chunks, straightforward repeated similarity work is approximately:

$$\boxed{O(NKd)}$$

subject to implementation details, caching, and precomputed similarities.

Example: $N = 100$, $K = 10$, $d = 384$:

$$100 \times 10 \times 384 = \boxed{384{,}000}.$$

For $d = 1536$:

$$100 \times 10 \times 1536 = \boxed{1{,}536{,}000}.$$

Thus, for the same candidate and selection sizes, quadrupling dimension from 384 to 1536 approximately quadruples raw coordinate-level arithmetic.

9 Embedding Geometry

For normalized vectors, cosine similarity becomes:

$$\cos(x, y) = x^\top y.$$

For approximately independent random unit vectors in d dimensions:

$$E[\cos(x, y)] \approx 0, \qquad Var[\cos(x, y)] \approx \frac{1}{d}.$$

Therefore:

$$\sigma \approx \frac{1}{\sqrt{d}}.$$

For $d = 384$:

$$\sigma \approx \frac{1}{\sqrt{384}} \approx 0.051.$$

For $d = 1536$:

$$\sigma \approx \frac{1}{\sqrt{1536}} \approx 0.0255.$$

This illustrates concentration in high-dimensional geometry: unrelated random vectors tend to concentrate more tightly around orthogonality. It does **not** mean that higher-dimensional embeddings automatically produce better retrieval or better MMR.

What matters is whether the embedding model creates a useful semantic geometry for both:

$$Sim(q, d) \quad \text{(relevance)}$$

and

$$Sim(d, d') \quad \text{(redundancy)}.$$

10 Optimization Strategies

Practical MMR implementations usually apply diversification only after first-stage retrieval:

$$\text{Large corpus} \rightarrow \text{Vector/Hybrid Search} \rightarrow \text{Top } N \rightarrow \text{MMR} \rightarrow \text{Top } K.$$

For normalized candidate embeddings arranged as $D \in \mathbb{R}^{N \times d}$, query similarities can be calculated using Dq. Candidate–candidate similarities can be precomputed with:

$$DD^\top.$$

Precomputation can reduce repeated similarity work, but requires:

$$O(N^2)$$

memory for the similarity matrix, making it appropriate for a candidate pool of tens or hundreds, not the entire enterprise corpus.

11 MMR vs. Neural Reranking

MMR operates on embedding-space similarities:

$$MMR(d) = \lambda Sim(q, d) - (1 - \lambda) \max_{d' \in S} Sim(d, d').$$

A neural reranker instead uses a learned scoring function:

$$(q, d) \rightarrow f_\theta(q, d) \rightarrow \text{relevance score.}$$

Therefore:

	MMR	Neural reranker
Primary purpose	Reduce redundancy	Improve relevance ranking
Core computation	Embedding similarity	Neural inference
Diversity	Explicit penalty	Not necessarily explicit
Typical cost	Relatively low	Higher

They can be combined:

$$\text{Vector Retrieval} \rightarrow \text{Top 50} \rightarrow \text{MMR} \rightarrow \text{Top 20} \rightarrow \text{Reranker} \rightarrow \text{Top 5} \rightarrow \text{LLM}.$$

Whether this helps should be tested against a baseline using metrics such as Context Precision, Context Recall, Faithfulness, Answer Relevancy, latency, and context/token cost.

12 Practical Enterprise Example

Suppose an enterprise corpus contains HR policies, an employee handbook, remote-work FAQs, security policies, and payroll documentation. For the query "What are the rules for employees working internationally?", naive retrieval may return several adjacent sections of the remote-work policy.

MMR may instead favor coverage across:

1. remote-work eligibility,
2. international restrictions,
3. manager approval,
4. tax/payroll implications,
5. security or compliance requirements.

This is where MMR can be valuable: overlapping enterprise knowledge sources and limited LLM context budgets.

13 Key Takeaways

1. MMR balances query relevance against redundancy.
2. It greedily selects chunks using relevance reward minus redundancy penalty.
3. λ controls the relevance–diversity trade-off.
4. MMR is useful when redundancy or overlapping evidence is a real problem.
5. MMR can hurt if diversification removes closely related evidence that a specific question genuinely needs.
6. Embedding dimension affects similarity-computation cost approximately linearly.
7. Straightforward greedy MMR has approximate similarity work of $O(NKd)$.
8. Higher dimension alone does not guarantee better diversification.
9. MMR and neural reranking solve different problems and should be evaluated empirically.

14 One-Sentence Summary

MMR selects context that is relevant to the query while penalizing chunks that repeat information already selected, helping a RAG system use a limited context budget more efficiently.

15 References

1. J. Carbonell and J. Goldstein, "The Use of MMR, Diversity-Based Reranking for Reordering Documents and Producing Summaries," SIGIR, 1998. `https://www.cs.cmu.edu/~jgc/publication/The_Use_MMR_Diversity_Based_LTMIR_1998.pdf`
2. Ragas Documentation. `https://docs.ragas.io/`
3. LangChain Documentation. `https://python.langchain.com/docs/`